

Relatório Técnico para o Desenvolvimento de uma Aplicação Baseada no Jogo Batalha Naval

Arthur Moss (1250771), Diogo Teixeira (1250873),
Pedro Nascimento (1251355), Tiago Santos (1251486)

Turma 1DA – Grupo 4

Abstract. This report serves as the basis for the development of a "Battleship" console game in C++ using Object-Oriented Programming (OOP). The application features local multiplayer and single-player modes, the latter including two difficulty levels. Key features include manual or automatic ship placement, real-time visual feedback, and a statistics system to track player performance and ranking. The project aims to provide an intuitive gaming experience through a simple terminal interface, serving as a guide for the application's implementation.

Keywords: C++, Object-Oriented Programming (OOP), Battleship, Console Application, Single-player and Multiplayer Modes, Game Statistics, and User Ranking.

1 Introdução

O objetivo deste projeto é desenvolver uma aplicação em C++ baseada no jogo Batalha Naval, utilizando, para isso, o conceito de programação orientada a objetos (OO), a aplicação deve correr diretamente na consola (terminal).

Este relatório servirá de base para o desenvolvimento da aplicação, uma vez que estabelece a arquitetura do sistema, detalha os requisitos funcionais e não funcionais e define o modelo de domínio necessário para garantir uma implementação estruturada e organizada.

Para além das funcionalidades principais do jogo, o projeto incluirá diferentes níveis de dificuldade, o registo das estatísticas dos jogadores para apresentação em rankings e menus de estatísticas, bem como uma interface simples e intuitiva, baseada em comandos de teclado.

2 Glossário

Tabela 1. Glossário

Term or Expression (EN)	Termo ou Expressão (PT)	Descrição (Definição e Informação)
Water	Água	Uma célula em que não se encontra nenhum navio ou um disparo que falhou o navio inimigo
Fleet	Frota	O conjunto dos 5 navios que cada jogador possui
Player	Jogador	A pessoa que está a participar na partida
Game	Jogo	Partida de Batalha Naval
Ship	Navio	Uma embarcação que ocupa várias células no tabuleiro
Orientation	Orientação	A posição do navio (vertical ou horizontal)
Cell	Célula	Cada uma das 100 células que formam o tabuleiro
Board	Tabuleiro	O mapa 10x10 onde se colocam navios e se marcam tiros
Shot	Tiro	Um disparo que acertou num navio inimigo
Victory	Vitória	O estado alcançado quando um jogador afunda toda a frota inimiga

3 Modelo de Domínio

O modelo de domínio (Fig. 1) é fundamental para a arquitetura do sistema, servindo como um mapa para organizar as interações entre as entidades do jogo. Fazer este modelo antes de começar a implementação do código ajuda a garantir que a lógica do jogo faça sentido e siga as regras da Programação Orientada a Objetos (OO).

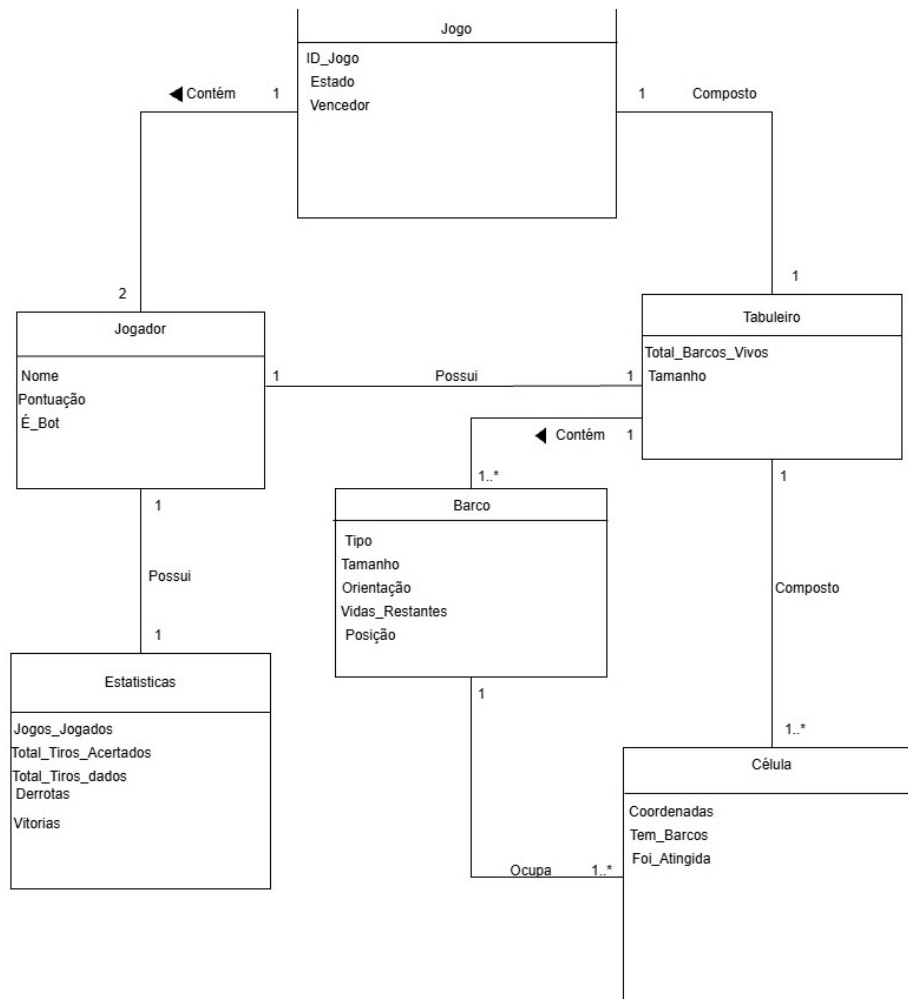


Fig. 1. Modelo de domínio que será utilizado no desenvolvimento da aplicação

4 Histórias do Utilizador

1. Como jogador, quero escolher entre jogar contra o computador ou contra outro jogador local, para adaptar a minha experiência de jogo.

Critérios de aceitação:

1. O sistema deve apresentar um menu inicial com as opções Jogador vs. Jogador e Jogador vs. Computador;
2. Ao seleccionar a opção Jogador vs. Jogador, o sistema deve pedir a introdução de dois utilizadores;

3. Ao selecionar a opção Jogador vs. Computador, o sistema deve iniciar a partida.
2. Como jogador, quero escolher o nível de dificuldade do jogo, para testar as minhas capacidades;

Critérios de aceitação:

1. A seleção da dificuldade deve estar disponível apenas no modo Jogador vs. Computador;
2. O sistema deve conter dois níveis de dificuldade:
 - Fácil: realiza disparos aleatórios;
 - Difícil: realiza disparos próximos de navios atingidos até afundá-los.
3. Como jogador, quero posicionar os meus navios no tabuleiro, para preparar a minha estratégia antes do jogo começar;

Critérios de aceitação:

1. O sistema deve pedir que o jogador introduza a coordenada de origem e a direção para cada navio;
2. O sistema deve validar se o navio encontra-se dentro dos limites do tabuleiro (10x10) e se não sobrepõe nenhum outro navio;
3. O sistema deve apresentar uma mensagem de erro caso o navio esteja fora dos limites do tabuleiro ou exista sobreposição de navios.
4. Como jogador, quero escolher as coordenadas de ataque, para destruir a frota inimiga;

Critérios de aceitação:

1. O sistema deve validar se a coordenada introduzida encontra-se dentro dos limites do tabuleiro e se não encosta nos outros navios;
2. O sistema deve impedir ataques em coordenadas já atingidas, solicitando outra coordenada;
3. Após cada tiro o sistema deve fornecer um feedback visual (tabuleiros) e textual (“Água”, “Atingido” ou Afundado”).
5. Como jogador, quero visualizar o estado atual de 2 tabuleiros, para acompanhar o progresso da partida e planejar os próximos ataques;

Critérios de aceitação:

1. O sistema deve exibir o tabuleiro do jogador, com as posições dos navios e ataques sofridos e um outro tabuleiro, com as posições atacadas pelo jogador.
6. Como jogador, quero consultar o meu histórico de estatísticas, para acompanhar a minha evolução no jogo;

Critérios de aceitação:

1. O sistema deve calcular e exibir o tempo de jogo total, número de vitórias/derrotas e a taxa de precisão.
7. Como jogador, quero visualizar o ranking, para comparar o meu desempenho com o de outros jogadores.

Critérios de aceitação:

1. O sistema deve guardar as estatísticas de cada jogador e exibir um ranking em ordem decrescente, dependendo da estatística selecionada.

5 Requisitos Funcionais

Descrevem as necessidades, funcionalidades e restrições que um sistema deve atender para satisfazer os utilizadores e objetivos do projeto. Divide-se em Requisitos Funcionais e Requisitos não Funcionais

5.1 Requisitos Funcionais

Descrevem as funções que o software deve realizar, ou seja, o que o sistema deve fazer em termos de entradas, processamento e saídas.

5.1.1 Especificação de Casos de Uso

Nesta secção apresenta-se a especificação detalhada dos casos de uso. Para cada caso de uso são indicadas a sua descrição, as pré-condições e pós-condições necessárias, bem como o caminho principal e, sempre que aplicável, os caminhos alternativos que descrevem situações de erro ou variantes do fluxo normal de interação.

UC 1. Registrar Jogador

Descrição	Adiciona um novo perfil ao sistema
Pré-condição	O nickname escolhido não deve existir no sistema
Pós-condição	O utilizador passa a ter um perfil no sistema
Caminho principal	1. O utilizador seleciona a opção “Não tenho perfil” 2. O sistema solicita o nickname 3. O utilizador insere o nickname 4. O sistema verifica se o nickname é único e cumpre o mínimo de 3 caracteres e o máximo de 24 caracteres, se não tem espaços e se não tem caracteres não ASCII 4.1. O sistema deteta que o nickname já existe, informa o erro e solicita um novo nickname 4.2. O sistema deteta que o tamanho nickname não está contido no intervalo de caracteres, informa o erro e solicita um novo nickname 5. O sistema armazena o nickname novo e retorna sucesso

UC 2. Registar com Perfil Existente

Descrição	Permite ao utilizador autenticar-se no sistema
Pré-condição	O nickname escolhido deve existir no sistema
Pós-condição	O utilizador fica autenticado no sistema
Caminho principal	1. O utilizador seleciona a opção “Tenho perfil” 2. O sistema solicita o nickname 3. O utilizador insere o nickname 4. O sistema verifica se o nickname existe 4.1. O sistema deteta que o nickname não existe, informa o erro e solicita o nickname novamente 5. O sistema abre o menu principal

UC 3. Consultar Ranking

Descrição	Exibe a tabela classificativa baseada no desempenho dos utilizadores
Pré-condição	Deve existir pelo menos um jogador registado
Pós-condição	O ranking é exibido
Caminho principal	1. O utilizador solicita a visualização do ranking 1.1. O sistema informa que ainda não existem dados para gerar o ranking e retorna ao menu principal 2. O sistema pergunta que tipo de ranking o utilizador quer visualizar, por exemplo: número de vitórias ou taxa de precisão 3. O sistema acede à base de dados de utilizadores 4. O sistema ordena os perfis, por ordem decrescente, com base no tipo de dados selecionado pelo utilizador 5. O sistema imprime o ranking 6. O sistema retorna ao menu principal

UC 4. Visualizar Estatísticas Individuais

Descrição	Exibe o histórico de estatísticas do jogador com sessão iniciada
Pré-condição	O utilizador deve estar no menu principal
Pós-condição	As estatísticas são imprimidas no ecrã
Caminho principal	1. O utilizador seleciona "Estatísticas" 2. O sistema lê os dados (vitórias, derrotas, tiros dados, tiros acertados, tiros falhados, tempo de jogo e taxa de precisão) do utilizador registrado 3. O sistema imprimir as estatísticas

UC 5. Consultar Ajuda e Regras

Descrição	Apresenta um manual rápido de como jogar e os símbolos utilizados no tabuleiro
Pré-condição	O utilizador deve estar no menu principal
Pós-condição	O utilizador visualiza as instruções
Caminho principal	1. O utilizador seleciona "Ajuda e Regras" 2. O sistema exibe a legenda dos símbolos (por exemplo: ~ = Água, # = Navio, X = Tiro certo, 0 = Tiro na água) 3. O sistema explica as regras do jogo 4. O utilizador regressa ao menu principal

UC 6. Começar Partida

Descrição	Define os parâmetros iniciais (modo e dificuldade) antes do início do jogo
Pré-condição	O utilizador deve estar no menu principal
Pós-condição	Os parâmetros da partida são definidos
Caminho principal	1. O utilizador seleciona "Iniciar Jogo" 2. O sistema solicita a escolha do modo (Jogador vs. Jogador ou Jogador vs. Computador) 3. O utilizador seleciona o modo Jogador vs. Computador 3.1. O utilizador seleciona o modo Jogador vs. Jogador 3.2. O sistema solicita o nickname do 2º jogador 3.3. O sistema confirma as definições e inicia o UC 7 4. O sistema solicita o nível de dificuldade (Fácil ou Difícil) 5. O sistema confirma as definições e inicia o UC 7

UC 7. Desenhar Tabuleiro

Descrição	Desenha a interface visual do jogo
Pré-condição	Existe uma partida ativa
Pós-condição	O tabuleiro é impresso para o utilizador
Caminho principal	1. O sistema limpa o terminal 2. O sistema imprime as letras A-J 3. O sistema percorre a matriz do tabuleiro e imprime o carácter ~ para água

UC 8. Posicionar Navios

Descrição	Permite ao utilizador colocar cada navio da sua frota no tabuleiro
Pré-condição	O UC 7 está terminado
Pós-condição	Todos os navios estão posicionados no tabuleiro
Caminho principal	1. O sistema imprime a lista de navios a posicionar e o tabuleiro 2. O utilizador selecciona o navio que quer posicionar 3. O utilizador indica a coordenada (ex: A1) e a orientação (Horizontal ou Vertical) 4. O sistema valida a posição (UC 10) 4.1. O sistema informa que o navio ultrapassa os limites do tabuleiro ou sobrepõe outro navio e solicita novos dados 5. O sistema atualiza o tabuleiro do utilizador 6. O sistema repete o processo até acabar a frota 6.1. Caso o utilizador esteja no modo Jogador vs. Jogador o sistema imprime um novo tabuleiro (UC 7) para o jogador 2 e repete o mesmo processo

UC 9. Posicionamento Automático de Navios

Descrição	O sistema coloca os navios de forma aleatória para o utilizador
Pré-condição	O modo de jogo já foi selecionado
Pós-condição	Todos os navios do utilizador ficam posicionados no tabuleiro
Caminho principal	1. O utilizador seleciona a opção "Posicionamento Automático" 2. O sistema gera coordenadas e orientações aleatórias para um dos navios da lista 3. O sistema valida a posição (usando as regras do UC 10) 4. O sistema repete o mesmo processo até que todos os navios estejam posicionados 5. O sistema imprime o tabuleiro finalizado ao utilizador

UC 10. Validar Posicionamento de Navio

Descrição	Verifica se um navio pode ser colocado numa determinada posição
Pré-condição	Coordenada, orientação e tipo de navio fornecidos
Pós-condição	O sistema retorna Válido ou Inválido
Caminho principal	1. O sistema verifica quantas célula o navio selecionado pelo utilizador ocupa 2. O sistema verifica se as células estão dentro da matriz (10x10) e se o navio não encosta em outros navios 2.1. Se o navio estiver fora dos limites do tabuleiro ou se encostar em outros navios, o sistema retorna "Inválido" e o motivo do erro 3. O sistema verifica se as células estão vazias 3.1. Se o navio se sobrepor a outro, o sistema retorna "Inválido" e o motivo do erro 4. O sistema retorna "Válido"

UC 11. Efetuar Jogada

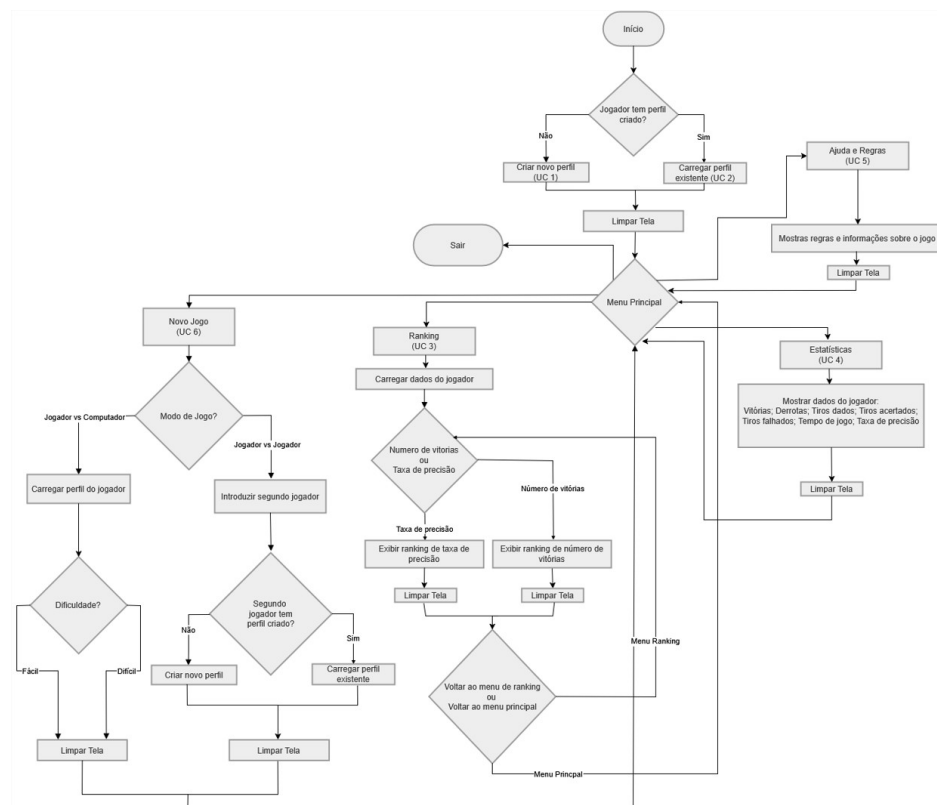
Descrição	O utilizador escolhe uma coordenada para atacar o adversário
Pré-condição	A partida já começou e é o turno do utilizador
Pós-condição	O estado da célula no tabuleiro adversário é alterado
Caminho principal	1. O sistema solicita uma coordenada de ataque 2. O utilizador insere a coordenada 2.1. O sistema avisa que a coordenada já foi atacada e pede uma nova coordenada 3. O sistema verifica o conteúdo da célula inimiga na coordenada indicada pelo utilizador 4. O sistema regista "Tiro" (se acertar) ou "Água" (se falhar) 5. O sistema atualiza o tabuleiro do e passa o turno 5.1. Caso o utilizador esteja no modo Jogador vs. Jogador, o sistema pergunta se é o jogador 2 que irá efetuar a jogada 6. O sistema limpa o terminal e imprime os tabuleiros do jogador

UC 12 - Verificar Condição de Vitória

Descrição	Analisa se o jogo deve terminar após um disparo
Pré-condição	Um disparo foi efetuado com sucesso (UC 11)
Pós-condição	O jogo termina ou continua para o próximo turno
Caminho principal	1. O sistema verifica se ainda restam partes de navios no tabuleiro atingido 2. O sistema confirma que todos os navios foram afundados 2.1. O sistema deteta que ainda há navios restantes e prossegue para o próximo turno 3. O sistema declara o vencedor e encerra a partida 4. O sistema imprime as estatísticas do jogador (Jogador vs. Computador) ou dos 2 jogadores (Jogador vs. Jogador) 5. O sistema atualiza as estatísticas individuais

5.1.2 Fluxograma

Com base nos casos de uso especificados anteriormente, foi elaborado um fluxograma (Fig. 2) que representa de forma visual o funcionamento geral da aplicação, desde o login do utilizador até ao final de uma partida.



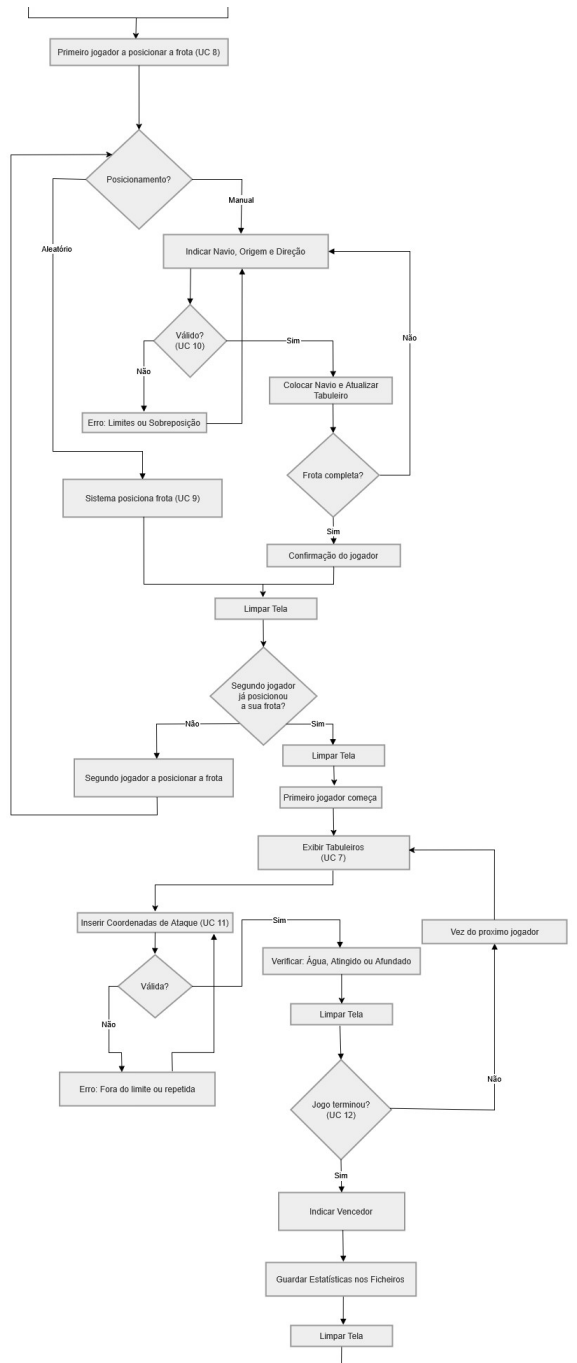


Fig. 2. Fluxograma que será utilizado no desenvolvimento da aplicação

5.2 Requisitos não Funcionais

Especificam propriedades do sistema ou restrições que melhoram a sua funcionalidade

5.2.1 Usabilidade

- Utilização de caracteres distintos e intuitivos para representação no tabuleiro: ~ (água), # (navio), X (navio atingido), 0 (tiro que acertou a água).
- O formato que o utilizador insere as coordenadas (por exemplo, "Linha, Coluna") é sempre o mesmo ao longo de todos os menus e rondas.
- Exibição de mensagens de erro claras quando o utilizador tenta disparar para uma coordenada que já foi atingida ou que está fora dos limites do tabuleiro.
- O tabuleiro impresso na consola deve ter as linhas e colunas sempre identificadas por letras de A a J nas linhas e números de 1 a 10 nas colunas.
- O sistema deve aceitar tanto minúsculas como maiúsculas, por exemplo escrever "A 1" ou "a 1" deve ser o mesmo e ambos funcionam.
- O sistema deve limpar o ecrã da consola entre cada ronda, exibindo apenas a informação relevante para ronda atual.

6 Arquitetura Física

A arquitetura física (Fig. 3) descreve o hardware necessária à execução da aplicação. Neste caso a aplicação apenas necessita de um computador onde a aplicação é executada e armazenada.

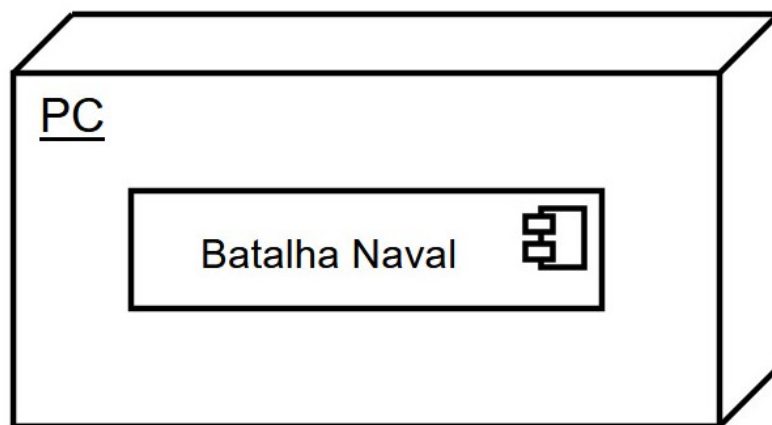


Fig. 3. Arquitetura física da aplicação

7 Arquitetura Lógica

A arquitetura lógica (Fig. 4) representa a forma como os diferentes componentes de software da aplicação se organizam e se relacionam entre si, seguindo os princípios da Programação Orientada a Objetos.

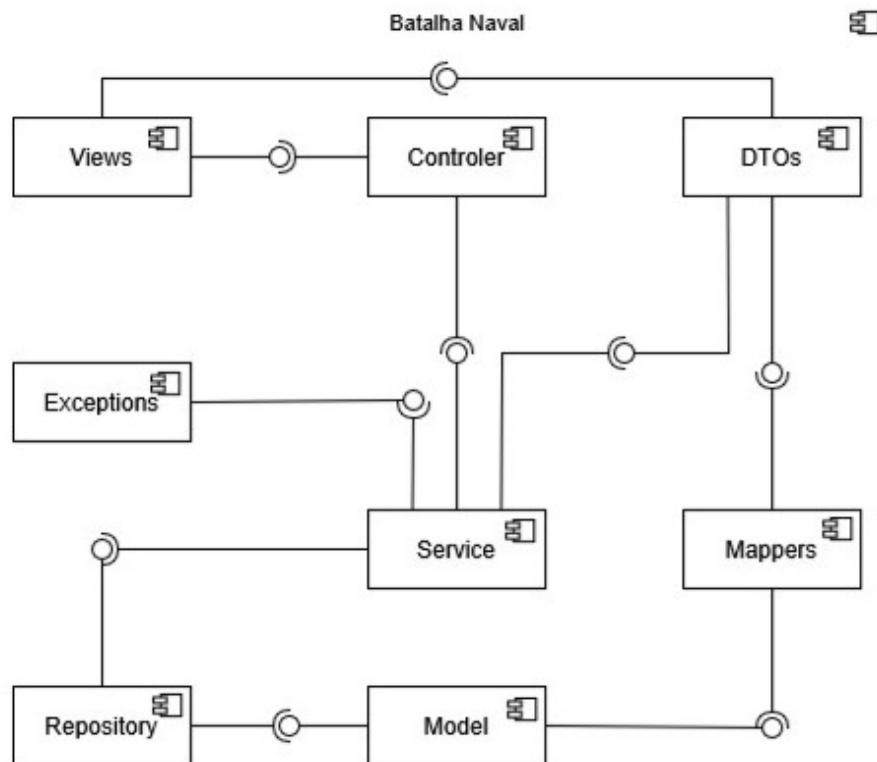


Fig. 4. Arquitetura lógica da aplicação

8 Organização do Código

De forma a complementar a arquitetura lógica apresentada anteriormente, a Fig. 5 mostra como o código da aplicação está organizado, assim como as dependências de inclusão existentes entre cada um.

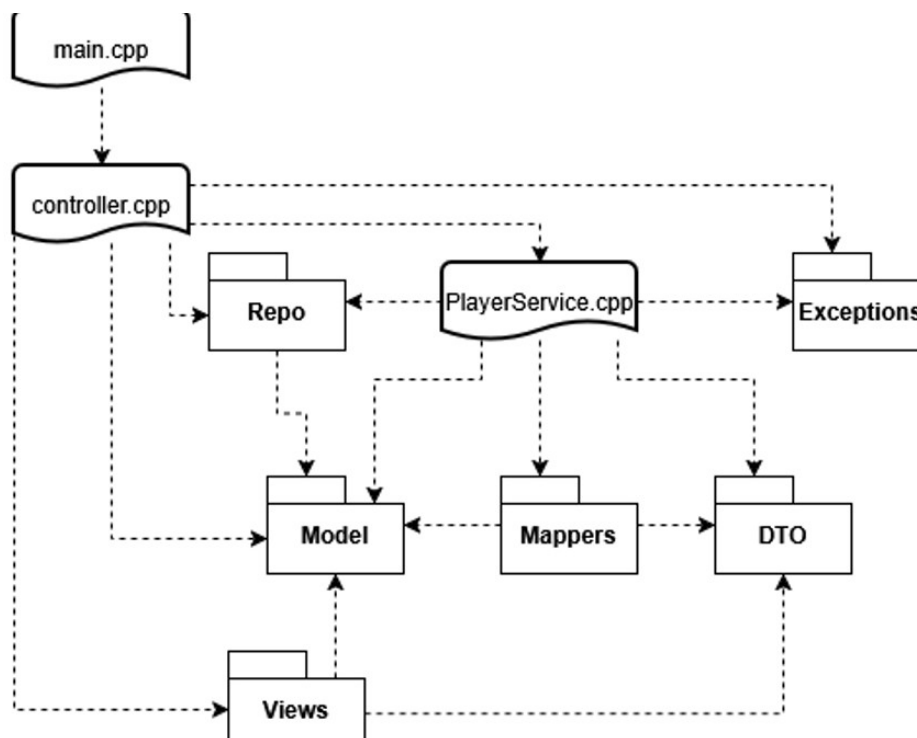


Fig. 5. Organização do código da aplicação

9 Diagrama de Classes de Modelo

O diagrama de classes (Fig. 6) de modelo representa as principais entidades do domínio da aplicação e a forma como se relacionam entre si.

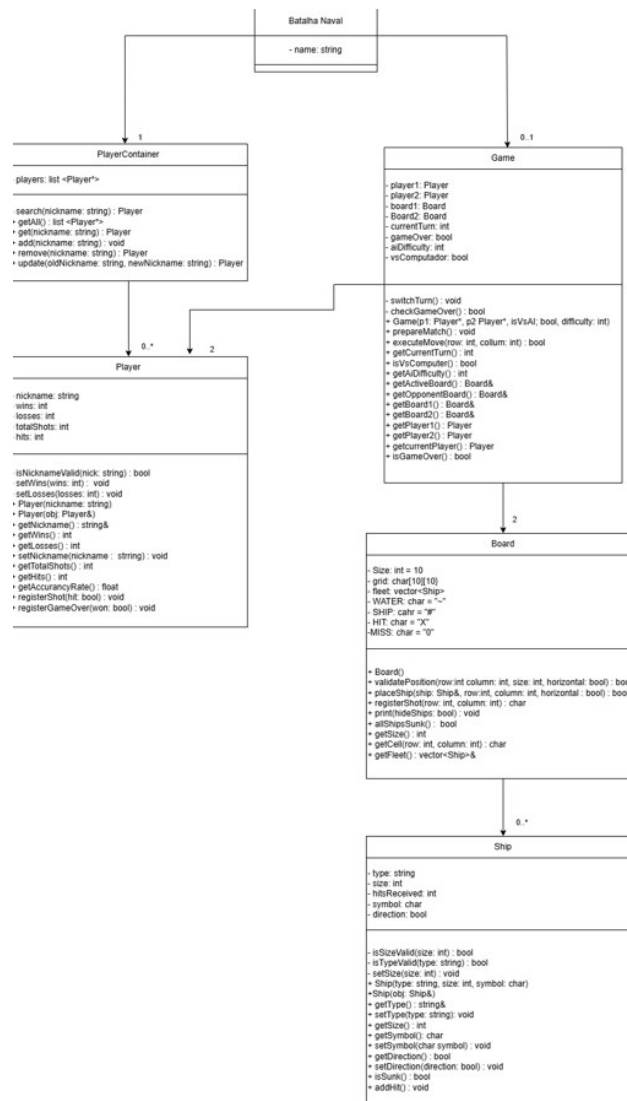


Fig. 6. Diagrama de classes de modelo que serão utilizadas na aplicação

10 Diagrama de Classes de Exceções

Para o tratamento de erros e a validação de dados, foram definidas três classes de exceções personalizadas: `DuplicatedDataException`, `InvalidDataException` e `NoDataException`. Estas derivam da classe `std::exception`. O diagrama seguinte (Fig. 7) apresenta estas classes, bem como os respectivos atributos e métodos.

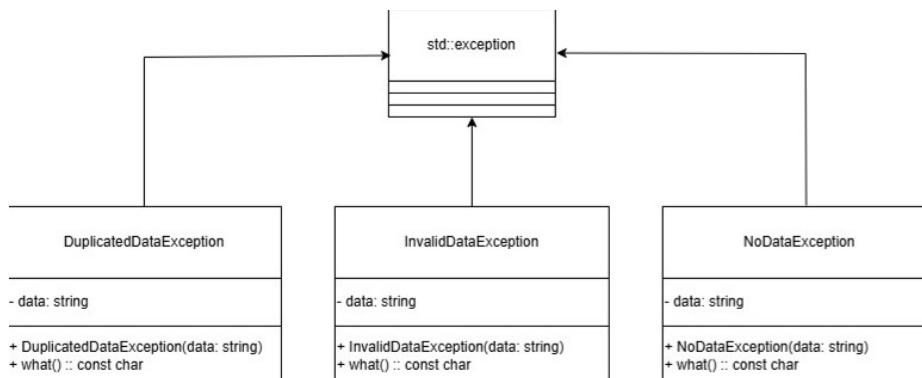


Fig. 7. Diagrama de classes de exceções que serão utilizadas na aplicação

11 Interface de Usuário

A interface de usuário (Fig. 8) mostra o funcionamento da aplicação do ponto de vista do usuário, isto é, a estrutura das interfaces pelas quais o utilizador irá navegar para aceder aos diferentes menus do sistema: menu de login, menu principal, novo jogo, dificuldade, posicionamento dos navios, tipo de ranking, estatísticas e ajuda e regras, e especificando as opções disponíveis em cada menu.

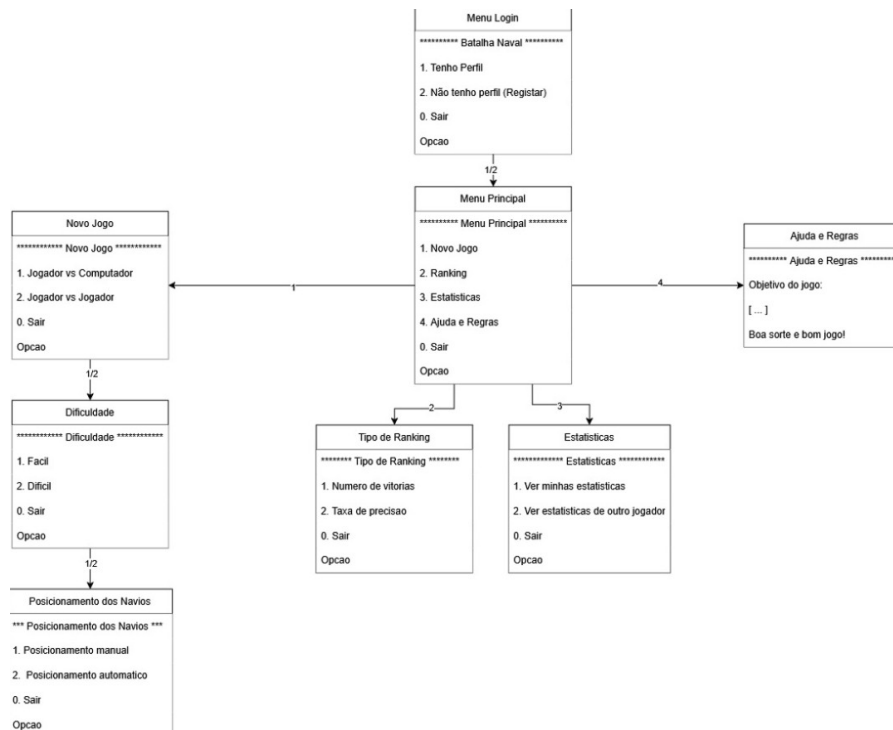


Fig. 8. Interface de usuário da aplicação

12 Implementação

12.1 DTO

```
struct PlayerDTO {  
    string nickname;  
    int victories;  
    int defeats;  
    int totalShots;  
    int shotsHit;  
    float accuracy;  
};
```

Fig. 9. Código de PlayerDTO.h

PlayerDTO é uma estrutura que transporta os dados do jogador entre camadas (Services, Controller e View). Apenas armazena campos como nickname, estatísticas e precisão. Isto evita que a lógica de domínio seja exposta à interface.

12.2 Exceptions

```
class InvalidDataException : public exception {  
private:  
    string data;  
public:  
    InvalidDataException(const string& data);  
    const char* what();  
};
```

Fig. 10. Código de InvalidDataException.h

```
InvalidDataException::InvalidDataException(const string& data){
    this->data = "\nERRO! " + data + " nao e valido!";
}

const char* InvalidDataException::what(){
    return data.c_str();
}
```

Fig. 11. Código de InvalidDataException.cpp

InvalidDataException é uma exceção personalizada que herda de `std::exception`, ela recebe uma mensagem de erro e o método `what()` e devolve essa mensagem, permitindo que o Controller mostre o erro ao utilizador de forma simples. As exceções `NoDataException` e `DuplicatedDataException` funcionam da mesma forma, mudando apenas a mensagem apresentada.

12.3 Model

```
class Board {
private:
    static const int SIZE = 10;
    char grid[SIZE][SIZE];
    vector<Ship> fleet;
    static const char WATER = '~';
    static const char SHIP = '#';
    static const char HIT = 'X';
    static const char MISS = '0';

public:
    Board();
    ~Board() = default;
    bool validatePosition(int row, int column, int size, bool horizontal) const;
    bool placeShip(const Ship& ship, int row, int column, bool horizontal);
    void placeShipAutomatically(int size, const string& type, char symbol);
    char registerShot(int row, int column);
    bool allShipsSunk() const;
    int getSize() const;
    char getCell(int row, int col) const;
    const vector<Ship>& getFleet() const;
    bool isWithinBounds(int row, int col, int size, bool horizontal) const;
    bool isCellPartOfShip(int row, int col, const Ship& ship) const;
};
```

Fig.12. Código de Board.h

```

bool Board::validatePosition(int row, int column, int size, bool horizontal) const {
    if (!isWithinBounds(row, column, size, horizontal))
        return false;

    int startRow = std::max(0, row - 1);
    int endRow = std::min(this->SIZE - 1, horizontal ? row + 1 : row + size);
    int startCol = std::max(0, column - 1);
    int endCol = std::min(this->SIZE - 1, horizontal ? column + size : column + 1);

    for (int i = startRow; i <= endRow; ++i) {
        for (int j = startCol; j <= endCol; ++j) {
            if (this->grid[i][j] != this->WATER) {
                return false;
            }
        }
    }

    return true;
}

```

Fig. 13. Código de validatePosition() em Board.cpp

Board representa o tabuleiro do jogo, com uma grelha de 10x10 onde cada posição pode conter água, navio, acerto ou falha. O método validatePosition() verifica se um navio pode ser colocado numa determinada posição, garantindo que não fica encostado a outros navios. Já os métodos placeShip() e registerShot() atualizam o estado do tabuleiro, enquanto allShipsSunk() verifica se todos os navios foram afundados e se o jogo terminou.

12.4 Mappers

```

class BoardMapper {
public:
    static void model2DTO(const Board& board, BoardDTO& dto);
};

```

Fig. 14. Código de BoardMapper.h


```

void BoardMapper::model2DTO(const Board& board, BoardDTO& dto) {
    dto.size = board.getSize();
    dto.allSunk = board.allShipsSunk();

    dto.grid.resize(dto.size, vector<char>(dto.size));
    for (int i = 0; i < dto.size; i++)
        for (int j = 0; j < dto.size; j++)
            dto.grid[i][j] = board.getCell(row: i, col: j);

    dto.fleet.clear();
    for (const Ship& ship : board.getFleet()) {
        ShipDTO boatDto;
        ShipMapper::model2DTO(ship, [&] boatDto);
        dto.fleet.push_back(boatDto);
    }
}

```

Fig. 15. Código de BoardMapper.cpp

BoardMapper converte um objeto Board num BoardDTO, copiando as informações necessárias para serem apresentadas na interface. Também utiliza o ShipMapper para converter os navios da frota. O método é static porque apenas realiza a conversão de dados e não precisa de guardar qualquer estado.

12.5 Repo

```
class ProfileRepositoryFile : public IProfileRepository {
private:
    static ProfileRepositoryFile* instance;

    PlayerContainer players;
    std::string filename;

    ProfileRepositoryFile();

    void load();
    void save();

public:
    static ProfileRepositoryFile* getInstance();
    PlayerContainer* getModel();

    bool exists(const std::string& nickname) override;
    void addProfile(const Player& player) override;
};
```

Fig. 16. Código de ProfileRepositoryFile.h

```
void ProfileRepositoryFile::save() {
    std::ofstream file(filename);
    if (!file.is_open()) {
        std::cerr << "ERRO: nao foi possivel guardar dados em " << filename << std::endl;
        return;
    }
    for (Player* p : players.getAll()) {
        file << p->getNickname() << ","
            << p->getWins() << ","
            << p->getLosses() << ","
            << p->getTotalShots() << ","
            << p->getHits() << "\n";
    }
    file.close();
}
```

Fig. 17. Código de save() em ProfileRepositoryFile.cpp

ProfileRepositoryFile é a classe responsável por guardar e carregar os perfis dos jogadores num ficheiro de texto (players.txt). Os métodos load() e save() permitem ler e guardar os dados dos jogadores, mantendo as suas estatísticas entre diferentes execuções do programa.

12.6 Services

```
class PlayerService {
private:
    PlayerContainer* container;

    PlayerDTO toDTO(const Player& player) const;

public:
    explicit PlayerService(PlayerContainer* container);
    ~PlayerService() = default;

    void registerPlayer(const PlayerLoginDTO& dto);
    bool playerExists(const string& nickname) const;
    PlayerDTO getPlayer(const string& nickname);
    list<PlayerDTO> getRankingByWins();
    list<PlayerDTO> getRankingByAccuracy();
};
```

Fig.18. Código de PlayerService.h

```
void PlayerService::registerPlayer(const PlayerLoginDTO& dto) {
    if (dto.nickname.empty()) {
        throw InvalidDataException("ERRO! Nickname nao pode estar vazio");
    }
    if (playerExists(dto.nickname)) {
        throw DuplicatedDataException("Jogador " + dto.nickname + " ja existe");
    }
    if (dto.nickname.length() < 3 || dto.nickname.length() > 24) {
        throw InvalidDataException("0 nickname deve ter entre 3 e 24 caracteres,");
    }
    container->add(dto.nickname);
}
```

Fig. 19. Código de registerPlayer() em PlayerService

PlayerService é a camada responsável pelas regras de negócio relacionadas com os jogadores. Faz validações no registo, verifica se já existem jogadores com o mesmo nome e obtém os rankings. Também organiza os jogadores com base nas vitórias ou na precisão. Em caso de erro, lança exceções específicas para que o Controller trate a situação adequadamente.

12.7 View

```
class View {
public:
    View();
    int menuMain();
    int menuNewGame();
    int menuDifficulty();
    int menuShipPlacement();
    int menuRankingType();
    int menuStatistics();
    int menuSatisfaction();
    void printHelpAndRules();
    void printMessage(const string& msg);
    void ShipPlacement(const string& type, int size, int& row, int& col, bool& horizontal);
    void showBoard(const Board& board, bool hideShips);
    void showPlacementSuccess();
    void showOutOfBoundsError();
    void showOverlapError();
    void showGameTurn(const string& player, const Board& yourBoard, const Board& opponentBoard);
    void showShotResult(bool hit, bool sunk, const string& shipType, const string& coordinate);
    void showGameOver(const GameDT0& game);
    bool getShotCoordinate(int& row, int& col);
};
```

Fig. 20. Código de View.h

```

void View::showBoard(const Board& board, bool hideShips) {
    cout << " ";

    for (int j = 0; j < 10; ++j) {
        cout << j << " ";
    }

    cout << "\n";

    for (int i = 0; i < 10; ++i) {
        cout << (char)('A' + i) << " ";

        for (int j = 0; j < 10; ++j) {
            char cell = board.getCell(row: i, col: j);

            if (hideShips && cell != '~' && cell != 'X' && cell != '0') {
                cout << "~ ";
            } else {
                cout << cell << " ";
            }
        }
        cout << "\n";
    }
}

```

Fig. 21. Código em showBoard() em View.cpp

View é a classe responsável pela interação com o utilizador através da consola. Mostra o tabuleiro, recebe as jogadas e apresenta mensagens e resultados do jogo. Também oculta os navios do adversário durante a partida e exibi as estatísticas finais quando o jogo termina.

12.8 Controller

```
class Controller {
private:
    View view;
    LoginView loginView;
    PlayerContainer* playerContainer;
    PlayerService* playerService;
    Game* currentGame;
    string currentNickname;

    void runNewGame();
    bool runPlacement(Board& board);
    void runRankingType();
    void runStatistics();
    void runHelpAndRules();
    void placeFleetManually(Board& board);
    void runGameLoop();

public:
    Controller();
    ~Controller();
    void runLogin();
    void runMain();
};
```

Fig. 22. Código de Controller.h

```

void Controller::runMain() {
    int op = -1;
    do {
        op = this -> view.menuMain();
        switch(op) {
            case 1:
                system( Command: "cls");
                runNewGame();
                break;
            case 2:
                system( Command: "cls");
                runRankingType();
                break;
            case 3:
                system( Command: "cls");
                runStatistics();
                break;
            case 4:
                system( Command: "cls");
                runHelpAndRules();
                break;
            default:
                break;
        }
    } while (op != 0);
    system( Command: "cls");
}

```

Fig. 23. Código de runMain() em Controller.cpp

Controller é a classe principal da aplicação, responsável por ligar a View, os Services e o Model. Controla o fluxo do programa, recebendo ações do utilizador, executando as regras de negócio e atualizando o estado do jogo. Além disso, organiza as diferentes funcionalidades em métodos separados.

13 Testes

Para garantir o correto funcionamento das classes, foram realizados testes unitários utilizando a framework Google C++ Testing Framework. Os testes feitos, cobrem construtores, funções e operadores de cada classe presente no projeto.

Segue-se um exemplo de teste realizado de cada classe.

Exemplo de teste para a classe Player:

Este teste verifica que o construtor da classe Player lança uma `InvalidDataException` quando o nickname introduzido pelo jogador, tem menos de 3 caracteres. Com isto, este teste assegura que a validação do nickname está corretamente implementada no projeto.

```
22  ✓ TEST(PlayerConstructorTest, NicknameTooShort) {  
23      bool flag = false;  
24  
25      try {  
26          Player p( nickname: "12");  
27      } catch (InvalidDataException& e) {  
28          flag = true;  
29      }  
30      EXPECT_TRUE(flag);  
31  }
```

Fig. 24. Teste unitário da classe Player – validação do nickname

Exemplo de teste para a classe Ship:

Este teste verifica que um navio é considerado afundado, após receber um número de hits igual ao seu tamanho. Neste caso, foram adicionados 2 hits, dado o tamanho da fragata. Este teste comprova que o método `isSunk()` reflete o estado do navio ao longo da partida.


```

34 ▶ TEST(ShipIsSunkTest, SunkAfterAllHits) {
35     Ship s( type: "Fragata", size: 2, symbol: '#');
36
37     s.addHit();
38     s.addHit();
39     EXPECT_TRUE(s.isSunk());
40 }

```

Fig. 25. Teste unitário da classe Ship – verificação de navio afundado

Exemplo de teste para a classe Board:

Este teste verifica que ao efetuar um disparo numa célula ocupada por um navio, neste caso novamente uma fragata, o método registerShot() devolve “X” e atualiza o estado do tabuleiro. Isto garante que ao longo da partida, após cada jogada, o tabuleiro é atualizado de forma correta.

```

57 ▶ TEST(BoardRegisterShotTest, ShotInShip) {
58     Board b;
59     Ship s( type: "Fragata", size: 2, symbol: '#');
60     b.placeShip(s, row: 0, column: 0, horizontal: true);
61
62     char result = b.registerShot(row: 0, column: 0);
63     EXPECT_EQ(result, 'X');
64 }

```

Fig. 26. Teste unitário da classe Board – registo de tiro no navio

Exemplo de teste para a classe Game:

Este teste verifica que quando um jogador acerta num navio do adversário, o turno não é alterado, permitindo assim que o jogador continue a jogar até falhar algum navio. Isto verifica que o projeto segue a regra do jogo clássico de batalha naval.

```

51 > TEST(GameExecuteMoveTest, HitKeepsTurn) {
52     Player p1(nickname: "pedro");
53     Player p2(nickname: "arthur");
54
55     Game g(&p1, &p2, isVsAI: false, difficulty: 0);
56     Ship s(type: "Fragata", size: 2, symbol: '#');
57
58     g.getBoard2().placeShip(s, row: 0, column: 0, horizontal: true);
59     g.executeMove(row: 0, column: 0);
60
61     EXPECT_EQ(g.getCurrentTurn(), 1);
62 }

```

Fig. 27. Teste unitário da classe Game – verificação de turno após acerto

Exemplo de teste para a classe PlayerContainer:

Este teste verifica que ao tentar encontrar um jogador com um nickname que não se encontra no sistema, é lançado um NoDataException. Isto assegura que o projeto trata de forma correta a gestão dos nicknames.

```

20 > TEST(PlayerContainerGetTest, GetNonExistent) {
21     PlayerContainer pc;
22     bool flag = false;
23
24     try {
25         pc.get(nickname: "jogadornaoexiste");
26     } catch (NoDataException& e) {
27         flag = true;
28     }
29
30     EXPECT_TRUE(flag);
31 }

```

Fig. 28. Teste unitário da classe PlayerContainer – Acessar jogador que não existe

13.1 Tabela de taxa de sucesso dos testes

A tabela seguinte apresenta os resultados obtidos na execução dos 34 testes unitários, indicando a taxa de sucesso de cada teste sobre as classes do projeto.

Tabela 2. Taxa de sucesso dos testes unitários realizados

Teste	Percentagem	Notas
Player — Construtor válido	100%	—
Player — Nickname curto demais	100%	—
Player — Nickname longo demais	100%	—
Player — registerShot (acerto)	100%	—
Player — registerShot (falha)	100%	—
Player — registerGameOver (vitória)	100%	—
Player — registerGameOver (derrota)	100%	—
Player — operator==	100%	—
Player — getAccuracyRate	100%	—
Ship — Construtor válido	100%	—
Ship — addHit	100%	—
Ship — isSunk (não afundado)	100%	—
Ship — isSunk (afundado)	100%	—
Ship — operator==	100%	—
Board — Construtor	100%	—
Board — placeShip válido	100%	—
Board — placeShip fora dos limites	100%	—
Board — placeShip sobreposição	100%	—
Board — registerShot em água	100%	—
Board — registerShot em navio	100%	—
Board — allShipsSunk (não afundado)	100%	—

Teste	Percentagem	Notas
Board — allShipsSunk (afundado)	100%	—
Board — getCell	100%	—
Board — placeShipAutomatically	100%	—
Game — Construtor	100%	—
Game — isVsComputer (J vs J)	100%	—
Game — isVsComputer (J vs Computador)	100%	—
Game — executeMove (falha)	100%	—
Game — executeMove (acerto)	100%	—
Game — isGameOver	100%	—
PlayerContainer — add e get	100%	—
PlayerContainer — get inexistente	100%	—
PlayerContainer — search existente	100%	—
PlayerContainer — search inexistente	100%	—

14 Conclusão

Esta secção apresenta as considerações finais do projeto, resumindo os resultados alcançados com o desenvolvimento da aplicação e o seu cumprimento dos requisitos funcionais e não funcionais.

14.1 Programação Orientada a Objetos

De forma geral, os princípios da Programação Orientada a Objetos foram bem seguidos.

As classes (Player, Ship, Board, Game) guardam os dados de forma privada e só permitem acesso através de métodos.

Os DTOs mostram apenas a informação essencial, escondendo a forma como ela é guardada internamente.

As heranças são usadas de forma simples e correta, as exceções herdam de exception. Não há herança entre Ship, Player, Board e Game.

Em resumo, os princípios foram bem aplicados ao longo de toda a implementação.

14.2 Cumprimento dos Requisitos Funcionais

Relativamente aos requisitos funcionais, todos os doze Casos de Uso foram implementados e seguidos. Os casos de uso abrangiam a gestão dos perfis dos utilizadores, as estatísticas, o início das partidas e a lógica do jogo.